

**SRIMATHI SUNDARAVALLI
MEMORIAL SCHOOL CHENNAI 600
063**

COMPUTER SCIENCE -QUESTION BANK –ANSWER KEY

1. Output Prediction Questions – Set 1

1. False (The .copy() method creates a shallow copy, resulting in a new dictionary object with a different ID).
2. [40, 50, 60, 70, 80]
50
3. [10, 20, 30, 40, 50, 60, 70]
[10, 20, 30, 50, 60, 70]
8
4. a) 60#35#
5. c) 34:31:30:31:

2. Block 2 Code Predictions

ChangeIt Problem: PPW%RR11N%

drawline Problem:

\$\$\$\$\$

@@@@@@@@@@

325

AAAAA

makenew Problem: The new string is : cBs eExAmS@2022cbs

L/D Stack Problem:

4*L

33*4

21*S

10*7

updatelist Problem:

[20, 3, 67, 89, 22]

[92, 3, 67, 100, 200, 300]

check Problem: [6, 7, 8, 9]

3. Block 3 Code Predictions

Execute / Output Problem:

0*11*12*3*

0*11*

0*11*12*

Global X / Change Problem:

10 # 25 \$ 185

18 # 50 \$ 318

30 # 100 \$ 480

4. Chapterwise Board Based Programs

Functions

1. Filter numbers with 3 or more digits:

```
L = [10, 100, 99, 200, 1000]
Newlist=[]
for I in L:
    if len(str(i))>3:
        Newlist.append(i)
print(Newlist)
```

2. SQUARE_LIST(L):

```
def SQUARE_LIST(L):
    SList = []
    for x in L:
        if x != 0:
            SList.append(x**2)
    return SList
```

3. isArmstrong(n):

```
def isArmstrong(n):
    temp = n
    digits = []
    while temp > 0:
        digits.append(temp % 10)
        temp //= 10

    power = len(digits)

    total_sum = 0
    for d in digits:
        total_sum += d ** power
```

```
    return total_sum == n

num = int(input("Enter number: "))
if isArmstrong(num):
    print(f'{num} is an Armstrong number')
else:
    print(f'{num} is not an Armstrong number')
```

4. secondLargest(L):

```
def secondLargest(L):
    largest = second = float('-inf')
    for num in L:
        if num > largest:
            second = largest
            largest = num
        elif num > second and num != largest:
            second = num
    return second
```

5. transform(L):

```
def transform(L):
    result = []
    for x in L:
        if x % 2 == 0:
            result.append(x**2)
        else:
            result.append(x**3)
    return result
```

6. Number Analysis:

```
def sumDigits(n):
    total = 0
    temp = n
    while temp > 0:
        total += temp % 10
        temp //= 10
    return total

def reverseNumber(n):
    rev = 0
    temp = n
```

```

while temp > 0:
    remainder = temp % 10
    rev = (rev * 10) + remainder
    temp //= 10
return rev

def isPalindrome(n):
    return n == reverseNumber(n)

num = int(input("Enter number: "))

print("Sum of digits =", sumDigits(num))
print("Reverse =", reverseNumber(num))

if isPalindrome(num):
    print("Palindrome = Yes")
else:
    print("Palindrome = No")

```

Stack

1. Travel Stack:

```

travel = []

def Push_element(NList):
    for city, country, dist in NList:
        if country != "India" and dist < 3500:
            travel.append([city, country, dist])

def Pop_element():
    while travel:
        print(travel.pop())
    print("Stack Empty")

NList = [ ["Newyork", "USA", 11734], ["Naypyidaw", "Myanmar", 3219],
["Dubai", "UAE", 2194], ["London", "England", 6693], ["Gangtok", "India", 1580],
["Columbo", "Sri Lanka", 3405]]

```

2. Races Stack:

```

stack = []

def Push(Races):
    for dist, name in Races.items():

```

```
        if dist > 100:
            stack.append(name)

def PoP():
    while stack:
        print(stack.pop())
Races ={100:'Varnika', 200 : 'Jugal', 400:'Kushal', 800:'Minisha'}}
```

Text Files

1. Word count per line:

```
def COUNT():
    with open('rhym.txt', 'r') as f:
        i = 1
        for line in f:
            words = line.split()
            print(f'Line {i} : {len(words)}')
            i += 1
```

2. Filter '@' lines:

```
def filter(oldfile, newfile):
    with open(oldfile, 'r') as fin, open(newfile, 'w') as fout:
        for line in fin:
            if not line.startswith('@'):
                fout.write(line)
```

3. Long lines:

```
def LongLines():
    with open('LINES.TXT', 'r') as f:
        for line in f:
            if len(line.split()) >= 10:
                print(line.strip())
```

4. Separate words by '#':

```
with open('sample.txt', 'r') as f:
    text = f.read()
```

```
words = text.split()
print('#'.join(words))
```

Binary Files (Pickle)

Restaurant Operations:

```
import pickle

def VegNonVeg(Rtype):
    with open('restaurant.dat', 'rb') as f:
        try:
            while True:
                data = pickle.load(f)
                for rno, details in data.items():
                    if details[1] == Rtype:
                        print(rno, details)
        except EOFError:
            pass

def UpdateRname(Rno, New_Name):
    records = {}
    with open('restaurant.dat', 'rb') as f:
        try:
            while True:
                records.update(pickle.load(f))
        except EOFError:
            pass
    if Rno in records:
        records[Rno][0] = New_Name
    with open('restaurant.dat', 'wb') as f:
        pickle.dump(records, f)

def DeleteRestaurant(Rno):
    records = {}
    with open('restaurant.dat', 'rb') as f:
        try:
            while True:
                records.update(pickle.load(f))
        except EOFError:
            pass
    records.pop(Rno, None)
    with open('restaurant.dat', 'wb') as f:
        pickle.dump(records, f)
```

```

def CountRestaurants(Rtype):
    count = 0
    with open('restaurant.dat', 'rb') as f:
        try:
            while True:
                data = pickle.load(f)
                for details in data.values():
                    if details[1] == Rtype:
                        count += 1
        except EOFError:
            pass
    return count

```

Test Record Operations:

```

def DisplayAvgMarks(Sub):
    total, count = 0, 0
    with open('TEST.dat', 'rb') as f:
        try:
            while True:
                rec = pickle.load(f)
                if rec[1] == Sub:
                    print(rec)
                    total += rec[3]
                    count += 1
        except EOFError:
            pass
    if count > 0:
        print("Average Marks:", total / count)

def CountFailed(Sub, PassMark):
    count = 0
    with open('TEST.dat', 'rb') as f:
        try:
            while True:
                rec = pickle.load(f)
                if rec[1] == Sub and rec[3] < PassMark:
                    count += 1
        except EOFError:
            pass
    return count

def ModifyMarks(TestId, New_ScoredMarks):
    recs = []
    with open('TEST.dat', 'rb') as f:

```

```

    try:
        while True:
            recs.append(pickle.load(f))
    except EOFError:
        pass
for rec in recs:
    if rec[0] == TestId:
        rec[3] = New_ScoredMarks
with open('TEST.dat', 'wb') as f:
    for rec in recs:
        pickle.dump(rec, f)

def RemoveTestRecord(TestId):
    recs = []
    with open('TEST.dat', 'rb') as f:
        try:
            while True:
                rec = pickle.load(f)
                if rec[0] != TestId:
                    recs.append(rec)
        except EOFError:
            pass
    with open('TEST.dat', 'wb') as f:
        for rec in recs:
            pickle.dump(rec, f)

```

CSV Files

Employee Payroll Operations:

```

import csv

def append_employees():
    with open('employee.csv', 'a', newline='') as f:
        writer = csv.writer(f)
        for _ in range(5):
            emp_id = input("EmpID: ")
            name = input("Name: ")
            dept = input("Department: ")
            salary = float(input("Salary: "))
            writer.writerow([emp_id, name, dept, salary])

def search_it_high_salary():
    with open('employee.csv', 'r') as f:
        reader = csv.reader(f)

```

```

        next(reader, None)
    for row in reader:
        if row and row[2] == "IT" and float(row[3]) > 50000:
            print(row)

def update_department(emp_id, new_dept):
    rows = []
    with open('employee.csv', 'r') as f:
        reader = csv.reader(f)
        for row in reader:
            if row and row[0] == emp_id:
                row[2] = new_dept
                rows.append(row)
    with open('employee.csv', 'w', newline='') as f:
        csv.writer(f).writerows(rows)

def delete_employee(emp_id):
    rows = []
    with open('employee.csv', 'r') as f:
        reader = csv.reader(f)
        for row in reader:
            if row and row[0] != emp_id:
                rows.append(row)
    with open('employee.csv', 'w', newline='') as f:
        csv.writer(f).writerows(rows)

def count_by_dept():
    counts = {}
    with open('employee.csv', 'r') as f:
        reader = csv.reader(f)
        next(reader, None)
        for row in reader:
            if row:
                dept = row[2]
                counts[dept] = counts.get(dept, 0) + 1
    print(counts)

```

Delivery Order Operations:

```

def DisplayCityOrders(City):
    with open('delivery.csv', 'r') as f:
        reader = csv.reader(f)
        for row in reader:
            if row and row[4] == City:
                print(row)

```

```

def CountPendingOrders():
    count = 0
    with open('delivery.csv', 'r') as f:
        reader = csv.reader(f)
        for row in reader:
            if row and row[3] == "Pending":
                count += 1
    return count

def MarkAsDelivered(OrderID):
    rows = []
    with open('delivery.csv', 'r') as f:
        reader = csv.reader(f)
        for row in reader:
            if row and row[0] == OrderID and row[3] in ["Shipped", "Pending"]:
                row[3] = "Delivered"
            rows.append(row)
    with open('delivery.csv', 'w', newline='') as f:
        csv.writer(f).writerows(rows)

def CancelOrder(OrderID):
    rows = []
    with open('delivery.csv', 'r') as f:
        reader = csv.reader(f)
        for row in reader:
            if row and row[0] != OrderID:
                rows.append(row)
    with open('delivery.csv', 'w', newline='') as f:
        csv.writer(f).writerows(rows)

```

5. Python Output Prediction – Practice Questions

Q1. Predict the output:

DT CEC

CEC

Q2. Predict the output:

P@OG@A**ING

2

Q3. Predict the output:

4

2

Q4. Predict the output:

Python-Programming

17

Q5. Predict the output:

False

True

True

Q6. Predict the output:

['X', 'Y', 'U', 'T', 'E', 'C', 'R']

XYUTECR

Q7. Predict the output:

[16, 4, 7, 9]

Q8. Predict the output:

[2, 4, 5]

[4, 2, 1]

Q9. Predict the output:

[10, 20, 30, 40]

[10, 20, 30, 40]

[10, 20, 30, 50]

Q10. Predict the output:

[2, 2, 4, 4, 6]

Q11. Predict the output:

[1, 3, 5, 7, 9]

Q12. Predict the output:

[10, 20, 30, 40, 50]

[99, 30, 20]

Q13. Predict the output:

(10, 99, 30, 20, 40)

1

Q14. Predict the output:

(3, 4, 1, 2, 3)

2

Q15. Predict the output:

5

[10, 15]

20

Q16. Predict the output:

{'A': 1, 'B': 20, 'C': 3}

('D', 4)

Q17. Predict the output:

{'A': [10, 20, 30], 'B': [60, 40]}

Q18. Predict the output:

{'B': 1, 'A': 3, 'N': 2}

['B', 'N']

Q19. Predict the output:

{'Maths': 95, 'CS': 100, 'English': 80}

275

Q20. Predict the output:

{3: 1, 1: 2, 4: 1, 5: 1}

[1, 3, 4, 5]

Revision 2

Q21. Predict the output:

[15, 10, 35, 20]

Q22. Predict the output:

{'A': 20, 'B': 30, 'C': 40}

Q23. Predict the output:

[1, 20, 30, 40, 4]

[99, 2, 3, 4]

Q24. Predict the output:

{'A': 1, 'B': 2, 'C': 3}

Q25. Predict the output:

True

13V31

2

Q26. Predict the output:

[5, 15, 30, 50]

Q27. Predict the output:

{'P': [1, 2, 3], 'Q': [3, 4, 7]}

Q28. Predict the output:

['ATAD', 'ECNEICS']

ATAD ECNEICS

Q29. Predict the output:

[2, 4, 6, 8, 2, 3, 4]

[2, 3, 4]

Q30. Predict the output:

{2: 6, 3: 6, 4: 4}

6

Q31. Predict the output:

[50, 20, 40]

(40, 20, 50)

Q32. Predict the output:

{'a': 5}

Q34. Predict the output:

30

[10, 50, 40, 50]

Q35. Predict the output:

('O', 'M', 'P', 'U', 'T', 'E')

{'O': 1, 'M': 1, 'P': 1, 'U': 1, 'T': 1, 'E': 1}

Q36. Predict the output:

[10, 30, 30]

{'A': 10, 'B': 20, 'C': 30, 'D': 70}

Q37

[1] [] {'x': 5, 'y': 10}

Q38

{'a': 1, 'b': 3, 'd': 5, 'c': 4}

Q39

('fourth', 4) ('third', 3) {'first': 1}

Q40

{'a': 3, 'b': 2}

Q41

('a', 4)

Q42

None Missing

None Missing

Q43

{'score': 70}

Q44

k3 v3 fallback

Q45

100 100 ('total', 100) {}

Q46

100 200 200 {}

{'a': 5}

6. Working with Functions – Advanced Practice

Q1: 15 10

Q2: 20 10 15

Q3: 52 / 32 / 14

Q4: [10, 20, 30] / [100, 20, 30, 40]

Q5: [100, 20, 30, 40]

Q6: 120

Q7: 7 5 3 1

Q8: 15 5

Q9: [1, 2, 5] / 15

Q10: (9, 20) / 29

Q11: 15 / 15

7. CBSE Cluster - 2024 Solutions

1. False (Order is try, except, else, finally)

2. a) two objects 2 references

3. c) <class 'function'>

4. b) `t.insert(2,3)`
5. c) `1 (4 // 3**2 = 4 // 9 = 0. Index of 'Sumit' is 1. 1 + 0 = 1)`
6. b) pickle
7. b) `math.ceil(19.22)+ math.floor(15.99) > 34 (20 + 15 = 35 > 34 is True)`
8. c) `CALC(y=25)` (Missing required positional parameter x)
9. Sunil
10. c) `S[::1][:2] + S[len(S)-1]`
11. d) `[6, 4, 2, '1', '0', '0']`
12. b) Both A and R are correct and R is not the correct explanation of A
13. a) `[student['first name'] for student in Data['students']]`
b) `len(Data['students']) + len(Data['teacher'])`
- c). Mutable objects can be modified in place without changing their identity (e.g., lists, dictionaries), whereas immutable objects cannot be changed once created (e.g., tuples, integers, strings).
- d). 0 (`max(False, -3, -4)` is 0, `min(0, 2, 7)` is 0)
14. a) `math.floor()`
b) `str.swapcase()`
15. Corrected Code:

```
L = [1,2,1,1,2,1,2,3,5,4,1,2,5,4,4,5]
D = {}
for x in L:
    if x not in D:
        D[x] = 1      # Corrected: initialized key
    else:
        D[x] += 1     # Corrected: increment count
print(D)
```

16. Outputs:

A Output:

`[1, None]`

`[1, None, 2, None]`

B Output:

`[1]`

`[1, 2]`

`[3]`

`[1, 2, 4]`

19. Unique Words Counter:

```
def countwords():
    with open("story.txt", "r") as f:
        words = f.read().split()
        unique = []
        for w in words:
            if w not in unique:
                unique.append(w)
        print(" ".join(unique))
```

20. Vowel Words Separator:

```
def count_vowels():
    with open("diary.txt", "r") as f:
        words = f.read().split()

        lower_vowels = []
        upper_vowels = []

        for w in words:
            if w[0] in 'aeiou':
                lower_vowels.append(w)
            elif w[0] in 'AEIOU':
                upper_vowels.append(w)

        print("List of words starting with lowercase vowels", lower_vowels)
        print("List of words starting with uppercase vowels", upper_vowels)
```

21. Garment Stack Operations:

```
stack = []
def Pushitems(Garment):
    for k, v in Garment.items():
        if v[1] > 5000:
            stack.append(v)

def Popitem():
    if not stack:
        print("Stack Empty")
    else:
        stack.pop()
        print("Remaining stack:", stack)
```

22. CSV Marks File Operations:

```

import csv

def Copycsv():
    with open('marks.csv', 'r') as fin, open('class.csv', 'w', newline='') as fout:
        writer = csv.writer(fout)
        for line in fin:
            name, cls, marks = line.strip().split(':')
            writer.writerow([name, marks])

def Copyselected():
    with open('marks.csv', 'r') as fin, open('class2.csv', 'w', newline='') as fout:
        writer = csv.writer(fout)
        for line in fin:
            parts = line.strip().split(':')
            if parts[1] == '2':
                writer.writerow(parts)

```

23. Binary Garment Operations:

a) CSV file can be opened with both Notepad and MS Excel.

```

import pickle

def Addgarment():
    with open('Garment.dat', 'ab') as f:
        gid = input("GID: ")
        gtype = input("Type: ")
        gender = input("Gender: ")
        cost = float(input("Cost: "))
        if gtype.lower() in ['cotton', 'silk']:
            pickle.dump([gid, gtype, gender, cost], f)

def Showgarment():
    count = 0
    with open('Garment.dat', 'rb') as f:
        try:
            while True:
                rec = pickle.load(f)
                if rec[1].lower() == 'silk':
                    print(rec)
                    count += 1
        except EOFError:
            pass
    print("Total count:", count)

```